

SQL Queries

Consider the following database about picture tagging Website:

Member(mid, name, age)

Picture(pid, year)

Tagged(mid, pid)

Relation Member contains a tuple for each registered member of the Website; Picture contains the metadata of the pictures (its key pid, and the year when the picture was taken), and Tagged says which user was tagged in what picture. Primary keys are underlined. Attributes values may be null.

- (a) Write a SQL query that retrieves the names of all members that were tagged both in the year 2011 and in 2014. Return them sorted in lexicographic order.

Solution:

```
select x.name
from Member x, Tagged y1, Picture z1, Tagged y2, Picture z2
```

```
where x.mid = y1.mid and y1.pid = z1.pid and z1.year = 2011
    and x.mid = y2.mid and y2.pid = z2.pid and z2.year = 2014
order by x.name;
```

Common mistakes:

1. Joining both Picture Tables to the same Tagged Table.

```
select M.name
from Member M, Tagged T, Picture P1, Picture P2
where M.mid = T.mid and P1.pid = T.pid and P2.pid = T.pid and
P1.year = 2011 and P2.year = 2014
order by M.name;
```

$T.pid = P1.pid$ and $T.pid = P2.pid \Rightarrow P1.pid = P2.pid \Rightarrow P1.year = P2.year$
since pid is the primary key of Picture

Students received 5 points for this answer.

2. Selecting Members who were tagged in 2011 or 2014 instead of both.

```
select M.name
from Member M, Tagged T, Picture P,
where M.mid = T.mid and P.pid = T.pid and
(P.year = 2011 or P.year = 2014)
order by M.name;
```

Students received 3 points for this answer.

- (b) Write a SQL query that returns for each member the total number of times that member was tagged in 2015. Order the results by the members' names. You should include all members, even those who were not tagged that year.

Solution:

```
select x.name, count(z.pid)
from Member x left outer join Tagged y on x.mid = y.mid
               left outer join Picture z on y.pid = z.pid and z.year = 2015
group by x.mid, x.name
order by x.name;
```

Common mistake:

```
SELECT m.name, COUNT(*) -- incorrect count
FROM Member m LEFT OUTER JOIN Tagged t ON (m.mid = t.mid), Picture p
               -- incorrect join
WHERE p.pid = t.pid and p.year=2015
group by m.name
order by m.name
```

Received 7 because of the incorrect join and needing a COUNT(p.pid)

```
SELECT m.name, COUNT(*)
FROM Member m, Tagged t, Picture p -- need outer join!
WHERE m.mid = t.mid and p.pid = t.pid and p.year=2015
group by m.name
order by m.name
```

Received 6 because no outer joining

A lot of students missed 1 point because they put count(*) instead of count(p.pid)

(c) Write a SQL query that returns the mid's and names of all members that were tagged only in pictures where Alice was also tagged.

```

select x.mid, x.name
from Member x
where x.mid not in
    (select y.mid
     from Tagged y
     where y.pid not in
         (select v.pid from Member u, Tagged v
          where u.name='Alice' and u.mid=v.mid));

```

Common mistakes:

1. Have a single level of negation.
2. Have a single level of negation combined with `u.name != 'Alice'`

Students received 4 points for such solutions.

(d) write a SQL query that retrieves the name of the members who were tagged in at least 3 photos after year 2022.

Solution:

Select x.name

From member x, Tagged y, Picture z

Where x.mid = y.mid and y.pid = z.pid and z.year > 2022

Group by x.mid, x.name

Having count(*) >= 3

(e) write a SQL query that retrieves pid of all the images where the people who are tagged in the photo have average age of 16 years old

```
Select z.pid
From member x, Tagged y, Picture z
Where x.mid = y.mid and y.pid = z.pid
Group by z.pid
Having avg(x.age) > 16
```

NOTE: we can only use subset of attributes in HAVING clause that are present in the GROUP BY clause. However, we can use any attribute that are not in the GROUP BY clause if they are in the aggregation function.

Indexes

Consider a workload consisting of many queries of this kind:

```
select x.name
from Member x, Tagged y, Picture z
where x.mid = y.mid and y.pid = z.pid and z.year = ?;
```

For each index below, indicate if it can potentially speed up the workload, if it is the only index that exists. You will assume that the Member and Tagged are very large relations, and that only a very small number of pictures are in any given year.

- | | |
|--|------------------|
| i. (1 point)
Index on <code>Picture(year)</code> . Yes or no? | i. <u>yes</u> |
| ii. (1 point)
Index on <code>Picture(pid)</code> . Yes or no? | ii. <u>no</u> |
| iii. (1 point)
Index on <code>Picture(pid,year)</code> . Yes or no? | iii. <u>no</u> |
| iv. (1 point)
Index on <code>Picture(year,pid)</code> . Yes or no? | iv. <u>yes</u> |
| v. (1 point)
Index on <code>Tagged(mid)</code> . Yes or no? | v. <u>no</u> |
| vi. (1 point)
Index on <code>Tagged(pid)</code> . Yes or no? | vi. <u>yes</u> |
| vii. (1 point)
Index on <code>Tagged(pid,mid)</code> . Yes or no? | vii. <u>yes</u> |
| viii. (1 point)
Index on <code>Member(mid)</code> . Yes or no? | viii. <u>yes</u> |
| ix. (1 point)
Index on <code>Member(name)</code> . Yes or no? | ix. <u>no</u> |
| x. (1 point)
Index on <code>Member(age)</code> . Yes or no? | x. <u>no</u> |

Relational Algebra

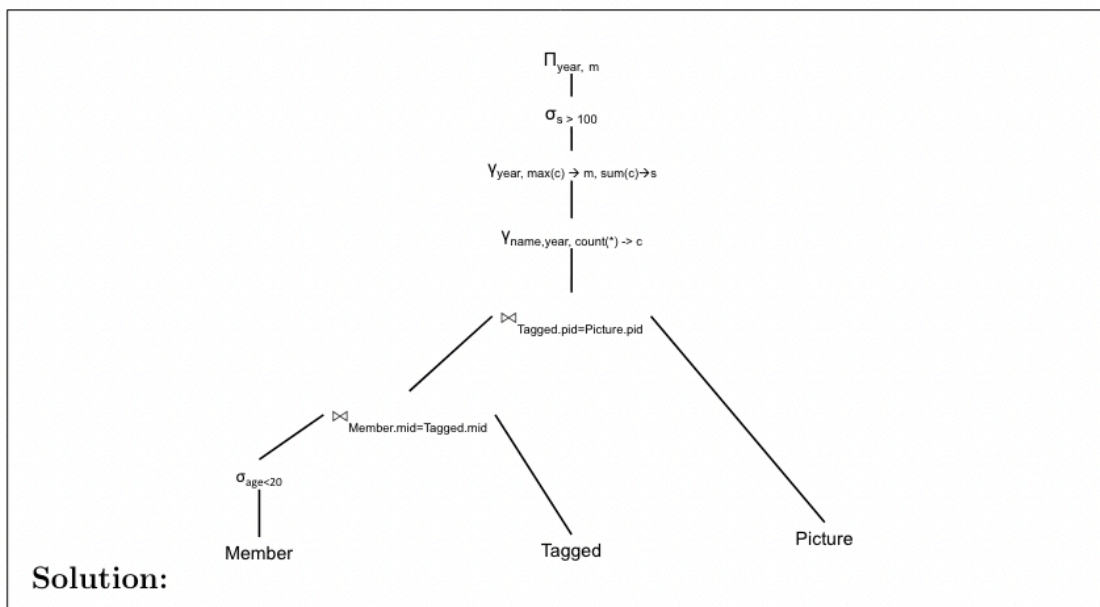
Consider the same relational schema as before: `Member(mid, name, age)`

`Picture(pid, year)`

`Tagged(mid, pid)`

Write a Relational Algebra expression in the form of a logical query plan (i.e., draw a tree) that is equivalent to the SQL query below. Your query plan does not have to be necessarily “optimal”: however, points will be taken off for overly complex solutions.

```
select w.year, max(w.c) as m
from (select x.name, z.year, count(*) as c
      from Member x, Tagged y, Picture z
      where x.mid = y.mid
            and y.pid = z.pid
            and age < 20
      group by x.name, z.year) w
group by w.year
having sum(w.c) > 100;
```



Consider below relations and answer to the relational algebra questions regarding these relations:

Student

sid	name	dept
001	Alice	CS
002	Bob	EE
003	Carol	CS
004	David	PHYS

Course

cid	title	dept	credits
CS425	Databases	CS	3
CS595	Database Security	CS	3
EE591	Microcomputers	EE	4
EE401	VLSI Design	EE	3
PHYS571	Radiation Physics	PHYS	3

Enroll

cid	sid	grade	grade point
CS425	001	A	4.0
CS595	001	B	3.0
CS595	002	A	4.0
EE401	001	A	4.0
EE401	002	B	3.0
EE401	004	A	4.0
PHYS571	002	C	2.0
PHYS571	004	A	4.0

Prereq

cid	pid
CS595	CS425
EE591	EE401
...	...

Write a relational algebra expression that returns the names of students that have taken all courses in their department.

Solution:

$$Q_{posscomb} = \pi_{sid, cid}(Student \bowtie Course)$$

$$Q_{actual} = \pi_{sid, cid}(Student \bowtie Enroll \bowtie Course)$$

$$Q_{all} = \pi_{sid}(Student) - \pi_{sid}(Q_{posscomb} - Q_{actual})$$

$$Q = \pi_{name}(Student \bowtie Q_{all})$$

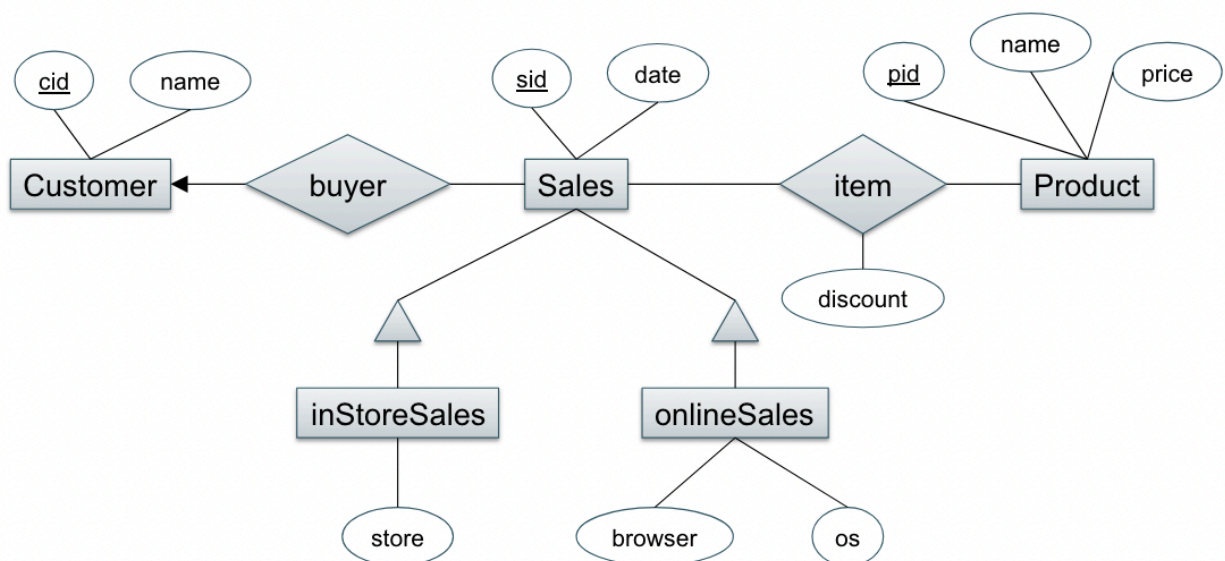
Write a relational algebra expression that returns the names of students that have gotten a better grade in a course than in at least one of the prerequisites of the course.

Solution:

$$Q_{better} = \pi_{sid}(\sigma_{grade > pgrade}(Enroll \bowtie Prereq \bowtie \rho_{pid,psid,pgrade}(\pi_{cid,sid,grade}(Enroll))))$$
$$Q = \pi_{name}(Student \bowtie Q_{better})$$

E/R Diagrams, Constraints, Conceptual Design

A company selling products both online and in their own stores has a database having a schema described by the E/R diagram below:



- Sales represents individual sales. One sale contains several products bought by one single customer. The discount is a real number representing a percent, for example discount = 33.33 means a discount of 1/3 from the original product price.
- inStoreSales consists of the sales in brick-and-mortar stores; each such sale contains the store name.

- onlineSales consists of online sales; each such sale includes the name of the browser and the operating system used during the purchase.
- cid, sid, pid are int, price and discount are float, the rest are text.
- No attributes may be null, except for inStoreSales.store and onlineSales.os.

(a) Create a database schema for the E/R diagram in the figure. You should turn an a set of CREATE TABLE statements. Your schema should include all constraints captured in the E/R diagrams

Solution:

```
drop table if exists Item;
drop table if exists onlineSales;
drop table if exists instoreSales;
drop table if exists Sales;
drop table if exists Product;
drop table if exists Customer;

create table Customer (cid int primary key, name text not null);
create table Product
    (pid int primary key,
     name text not null,
     price float not null);
create table Sales (
    sid int primary key,
    date text not null,
    cid int not null references customer);
create table inStoreSales (
    sid int primary key references Sales,
    store text);
create table onlineSales (
    sid int primary key references Sales,
    browser text not null,
    os text);
create table item (
    pid int references Product,
    sid int references Sales,
    discount float not null,
    primary key (pid, sid));
```

- (b) Write a SQL query that computes, for each customer, the total amount that the customer spent online. Your query does not need to include customers who did not purchase anything online.

Solution:

```
select v.cid, v.name, sum(u.price * (100-z.discount))/100
from onlineSales x, Sales y, Item z, Product u, Customer v
where x.sid = y.sid
    and y.sid = z.sid
    and z.pid = u.pid
    and y.cid = v.cid
group by v.cid, v.name;
```

Normal Forms

Consider the following relational schema and set of functional dependencies.

R(A,B,C,D,E,F,G) with functional dependencies:

$E \rightarrow C$

$G \rightarrow AD$

$B \rightarrow E$

$C \rightarrow BF$

- (a) Give one example of non-trivial functional dependency implied by the ones above:

Solution: Many solutions are possible. One example is $E \rightarrow BF$.

- (b) Compute E^+ , the closure of E.

Solution: $\{E\}^+ = \{B, C, E, F\}$

(c) Why do we bother to decompose relations into BCNF? What does this normal form ensure?

Solution:

BCNF ensures that, for each table, only “good” FDs exist: Whenever $X \rightarrow B$ is a non-trivial dependency, then X is a super key. Because only these good FDs exist, the relation will not suffer from data redundancy, update anomalies, nor deletion anomalies

(d) Decompose R into BCNF. Show your work for partial credit. Your answer should consist of a list of table names and attributes and an indication of the keys in each table (underlined attributes).

Solution: $\{E\}^+ \rightarrow \{B, C, E, F\}$ violates BCNF because it is neither trivial nor contains all attributes (i.e., E is not a superkey). So we need to decompose. We decompose into $R1(B, C, \underline{E}, F)$ and $R2(A, D, \underline{E}, \underline{G})$.

For $R1(B, C, \underline{E}, F)$, there are no more FD that violate BCNF, so we continue with $R2(A, D, \underline{E}, \underline{G})$. We find $\{G\}^+ = \{A, D\}$, so we decompose into $R21(\underline{G}, A, D)$ and $R22(\underline{G}, \underline{E})$. $R22$ has now two attributes, so it is necessarily in BCNF. For $R21$, there are no more FDs that violate BCNF.

True/False

Hash indexes benefit range selection queries

(False)

A relation can have at most one unique key.

(False)

Every key is a superkey.

(True)

A given schema with a set of functional dependencies can have multiple minimal superkeys.

(True)

If a table $R(A, B, C)$ already has a B+-tree index created on $R(A, B)$, then there is no benefit in creating another B+-tree index on $R(B)$, because any query that can be sped up by the second index can be done just as fast using the first index

(False)

Given two relations $R(A, B)$ and $S(C, D)$ both without any nulls, the following equality holds

$$R - \pi_{AB} (R \bowtie_{B=C} S) = \pi_{AB} (R \bowtie_{B \neq C} S)$$

(True)

Given $R(A,B,C,D,E)$, if $A \rightarrow C$, $AB \rightarrow D$, $AED \rightarrow B$, then $\{A,B,C,D,E\}$ is not the closure of AB .

(True)

The right-hand side Y (which can be one or more attributes) of a completely non-trivial FD $X \rightarrow Y$ cannot be a super key.

(False)